

```
In [1]: import pandas as pd
import numpy as np
import re

# NLP
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# ML
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Download NLTK data
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Chint\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Chint\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

Out[1]: True

```
In [2]: data = {
    "candidate": [
        "Rahul",
        "Priya",
        "Arjun",
        "Sneha"
    ],
    "resume": [
        "Python Java SQL Machine Learning Data Analysis",
        "JavaScript React HTML CSS Node.js",
        "Python Deep Learning NLP TensorFlow",
        "Java SQL Spring Boot Hibernate"
    ]
}

df = pd.DataFrame(data)

print(df)
```

	candidate	resume
0	Rahul	Python Java SQL Machine Learning Data Analysis
1	Priya	JavaScript React HTML CSS Node.js
2	Arjun	Python Deep Learning NLP TensorFlow
3	Sneha	Java SQL Spring Boot Hibernate

```
In [3]: job_description = """
Python Machine Learning SQL Data Analysis NLP
"""
```

```
print(job_description)
```

Python Machine Learning SQL Data Analysis NLP

```
In [4]: stop_words = set(stopwords.words('english'))
```

```
def clean_text(text):  
    text = text.lower()  
  
    text = re.sub(r'^a-zA-Z ', '', text)  
  
    words = word_tokenize(text)  
  
    filtered_words = [  
        word for word in words  
        if word not in stop_words  
    ]  
  
    return " ".join(filtered_words)
```

```
stop_words = set(stopwords.words('english'))
```

```
def clean_text(text):  
    text = text.lower()  
  
    text = re.sub(r'^a-zA-Z ', '', text)  
  
    words = word_tokenize(text)  
  
    filtered_words = [  
        word for word in words  
        if word not in stop_words  
    ]  
  
    return " ".join(filtered_words)
```

```
In [5]: df["clean_resume"] = df["resume"].apply(clean_text)
```

```
clean_job = clean_text(job_description)
```

```
print(df[["candidate", "clean_resume"]])
```

	candidate	clean_resume
0	Rahul	python java sql machine learning data analysis
1	Priya	javascript react html css nodejs
2	Arjun	python deep learning nlp tensorflow
3	Sneha	java sql spring boot hibernate

```
In [6]: vectorizer = TfidfVectorizer()
```

```
resume_vectors = vectorizer.fit_transform(df["clean_resume"])
```

```
job_vector = vectorizer.transform([clean_job])
```

```
In [7]: scores = cosine_similarity(job_vector, resume_vectors)

print(scores)
```

```
[[0.85761953 0.          0.44966943 0.12460461]]
```

```
In [8]: df["score"] = scores[0]

print(df[["candidate", "score"]])
```

	candidate	score
0	Rahul	0.857620
1	Priya	0.000000
2	Arjun	0.449669
3	Sneha	0.124605

```
In [9]: ranked_df = df.sort_values(by="score", ascending=False)

print(ranked_df[["candidate", "score"]])
```

	candidate	score
0	Rahul	0.857620
2	Arjun	0.449669
3	Sneha	0.124605
1	Priya	0.000000

```
In [10]: skills = [
    "python",
    "java",
    "sql",
    "machine learning",
    "nlp",
    "react",
    "tensorflow",
    "data analysis"
]
```

```
In [11]: def extract_skills(text):

    text = text.lower()

    found_skills = []

    for skill in skills:
        if skill.lower() in text:
            found_skills.append(skill)

    return found_skills

df["skills_found"] = df["resume"].apply(extract_skills)

print(df[["candidate", "skills_found"]])
```

	candidate	skills_found
0	Rahul	[python, java, sql, machine learning, data ana...
1	Priya	[java, react]
2	Arjun	[python, nlp, tensorflow]
3	Sneha	[java, sql]

In [12]: `required_skills = extract_skills(job_description)`

```
def missing_skills(candidate_skills):

    missing = []

    for skill in required_skills:
        if skill not in candidate_skills:
            missing.append(skill)

    return missing

df["missing_skills"] = df["skills_found"].apply(missing_skills)

print(df[["candidate", "missing_skills"]])
```

	candidate	missing_skills
0	Rahul	[nlp]
1	Priya	[python, sql, machine learning, nlp, data anal...
2	Arjun	[sql, machine learning, data analysis]
3	Sneha	[python, machine learning, nlp, data analysis]

In [14]: `df["skills_found"] = df["resume"].apply(extract_skills)`

In [15]: `df["missing_skills"] = df["skills_found"].apply(missing_skills)`

In [16]: `ranked_df = df.sort_values(by="score", ascending=False)`

```
In [17]: for index, row in ranked_df.iterrows():

    print("Candidate:", row["candidate"])

    print("Score:", round(row["score"], 2))

    print("Skills Found:", row["skills_found"])

    print("Missing Skills:", row["missing_skills"])

    print("-" * 40)
```

Candidate: Rahul
Score: 0.86
Skills Found: ['python', 'java', 'sql', 'machine learning', 'data analysis']
Missing Skills: ['nlp']

Candidate: Arjun
Score: 0.45
Skills Found: ['python', 'nlp', 'tensorflow']
Missing Skills: ['sql', 'machine learning', 'data analysis']

Candidate: Sneha
Score: 0.12
Skills Found: ['java', 'sql']
Missing Skills: ['python', 'machine learning', 'nlp', 'data analysis']

Candidate: Priya
Score: 0.0
Skills Found: ['java', 'react']
Missing Skills: ['python', 'sql', 'machine learning', 'nlp', 'data analysis']

In []: